When an event is persisted, the event handler (shown in the next listing) adds the last piece of the puzzle. The actor reacts differently depending on its current state and the processing event, often updating its own state in different ways.

Listing 12.5 Event handler and event

```
object Market {
  ...

  private def handleEvents(state: State, event: Event): State = {
    (state, event) match {
      case (_, Opened(marketId, fixture, odds)) =>
        OpenState(Status(marketId, fixture, odds, 0))          ❶
      case (state: OpenState, Updated(_, odds, result)) =>
        state.copy(status = Status(                            ❶
          state.status.marketId,
          state.status.fixture,
          odds.getOrElse(state.status.odds),
          result.getOrElse(state.status.result)))
      case (state: OpenState, Closed(_, result, _)) =>
        ClosedState(state.status.copy(result = result))        ❶
      case (_, Cancelled(_, _)) =>
        CancelledState(state.status)                           ❶
  }
  ...
}
```

❶ Each time events are processed, different states are produced.

This brings us full circle: the actor receives a command that can trigger an event that can change the state, and this happens over and over again for each message.

With these three parts—the protocol, the command handler that connects states and commands, and the event handler that connects events and states—you have a complete picture of the actor's behavior. But since the devil is in the details, let's look at the rest of the command handler methods of the market actor.